Real-Time Violence Detection in Surveillance Streams

Avi Verma
Dept. of Applied Mathematics
Delhi Technological University
New Delhi, India
aviverma_mc22a13_59@dtu.ac.in, vavi3984@gmail.com

***Abstract*—The escalating threat of violence in public spaces necessitates scalable, automated, and real-time detection systems to enhance urban safety. This study proposes a deep learning-based framework for real-time violence detection, utilizing a fine-tuned DenseNet121 convolutional neural network (CNN) optimized for processing Real-Time Streaming Protocol (RTSP) surveillance streams. Trained on a curated subset of the UCF-Crime dataset comprising 1,000 labeled frames, the model achieves a validation accuracy of 92% and a weighted F1-score of 0.91. Techniques such as data augmentation (e.g., rotation, zoom, shear) and class weight balancing mitigate class imbalance and enhance generalization. The system integrates OpenCV for efficient frame capture, Flask for real-time dashboard visualization, MongoDB for metadata logging, and Dropbox for secure cloud storage. A multithreaded architecture ensures concurrent processing of multiple RTSP streams, maintaining an average performance of $30\,\mathrm{fps}$ on a T4 GPU. This end-to-end pipeline demonstrates scalability, robustness, and deployability, offering a practical solution leveraging accessible hardware and open-source tools for smart city surveillance, transportation hubs, and institutional security. By combining deep learning with cloud-aware infrastructure, this work contributes a framework for public safety agencies to enable rapid incident detection and response.**

***Index Terms*—Violence Detection, DenseNet121, Real-Time Surveillance, Deep Learning, Multithreading, Cloud Integration**

I. Introduction

The proliferation of surveillance systems in urban environments underscores the urgent need for automated violence detection to ensure public safety. Manual monitoring of CCTV footage is labor-intensive, error-prone, and incapable of real-time response, necessitating intelligent systems that can analyze video streams autonomously. Traditional methods, such as Histogram of Oriented Gradients (HOG) with Support Vector Machines (SVM) [1], optical flow [2], and handcrafted spatiotemporal features, suffer from limited scalability and poor robustness in diverse real-world conditions. In contrast, deep learning approaches, particularly convolutional neural networks (CNNs) and recurrent models like Long Short-Term Memory (LSTM) networks [3], have demonstrated superior performance by learning complex patterns directly from raw data [4]–[6].

Despite these advances, existing deep learning-based violence detection systems often face challenges in achieving real-time performance, processing multiple streams concurrently, or integrating with scalable cloud infrastructure. Many models are computationally intensive, requiring high-end hardware, or lack practical deployment strategies for resource-constrained environments. This paper addresses these gaps by proposing a robust, real-time violence detection system based on a fine-tuned DenseNet121 CNN, optimized for RTSP streams and integrated with a cloud-aware, multithreaded architecture.

The proposed system leverages DenseNet121's dense connectivity for efficient feature extraction, achieving high accuracy while maintaining a lightweight footprint suitable for real-time inference. It incorporates OpenCV for frame capture, Flask for web-based visualization, MongoDB for metadata management, and Dropbox for secure cloud storage. A multithreaded pipeline ensures concurrent processing of multiple streams, achieving $30\,\mathrm{fps}$ on a T4 GPU. The system is packaged as a standalone executable using PyInstaller, enhancing portability and ease of deployment.Additionally, the ethical implications of video-based surveillance are considered, particularly with regard to privacy concerns, potential bias in training data, and the importance of transparency in automated decision-making systems.

Key Contributions of this work include:

- A fine-tuned DenseNet121 model for accurate binary classification of violence/non-violence in surveillance footage, achieving 92% accuracy and 0.91 F1-score.
- A multithreaded architecture for real-time processing of multiple RTSP streams with thread-safe frame management.

- Seamless integration with cloud services (MongoDB, Dropbox) and a Flask-based dashboard for scalable storage and visualization.
- A portable, standalone executable for deployment on mid-range hardware, supporting edge and cloud-based surveillance scenarios.

The paper is organized as follows: Section II reviews related work. Section III details the methodology, including dataset, preprocessing, model architecture, and system workflow. Section IV presents experimental results. Section V analyzes strengths, limitations, and implications. Section VI concludes the paper, and Section VII acknowledges contributors.

II. Related Work

A. Traditional Approaches

Early violence detection systems relied on hand-engineered features. Dalal and Triggs [1] introduced HOG for human detection, later adapted for violence detection by extracting motion patterns. Optical flow methods [2] modeled temporal dynamics but struggled with noise, occlusion, and lighting variations. Spatiotemporal features, such as Space-Time Interest Points (STIP) [7], improved robustness but required extensive feature engineering, limiting scalability in diverse surveillance scenarios.

B. Deep Learning-Based Methods

Deep learning has revolutionized violence detection. CNNs like VGG16 [4], ResNet50 [5], and InceptionV3 [8] excel at frame-level classification by learning hierarchical features. Temporal modeling with LSTMs [3] or 3D CNNs [9] captures action sequences, improving detection of dynamic events. DenseNet121 [6], with its dense connectivity, reduces parameters and mitigates vanishing gradients, making it ideal for transfer learning on smaller datasets like violence detection.

C. Real-Time and Edge-Based Systems

Real-time violence detection is critical for surveillance. Soleimani et al. [10] proposed lightweight CNNs for edge devices, but their models lacked multistream processing. Zhou et al. [11] used multi-region R-CNNs for action detection, achieving real-time performance but requiring high computational resources. Cloud integration, as explored by Chen et al. [12], enhances scalability but often overlooks real-time constraints.

D. Positioning of This Work

This study bridges the gap between academic models and practical deployment by combining DenseNet121's efficiency with a multithreaded, cloud-integrated pipeline. Unlike prior work, it supports concurrent RTSP stream processing, real-time visualization, and scalable storage, making it suitable for urban surveillance applications.

III. Methodology

This section details the dataset, preprocessing, model architecture, system workflow, training configuration, and deployment strategy for the proposed real-time violence detection system.

A. Dataset

The system utilizes a curated subset of the UCF-Crime dataset [13], comprising 1,000 anonymized video frames (500 for training, 500 for validation). The dataset has a class distribution of approximately 60% non-violent (600 frames) and 40% violent (400 frames), reflecting real-world imbalance. Frames are resized to 224×224 pixels and anonymized by blurring faces and removing identifiable features using standard image processing techniques, eliminating the need for Institutional Review Board (IRB) approval. Figure 1 shows representative frames.

Fig. 1. Representative frames from the UCF-Crime dataset: non-violent (left) and violent (right), resized to 224×224 pixels, anonymized to comply with ethical standards.

B. Data Preprocessing

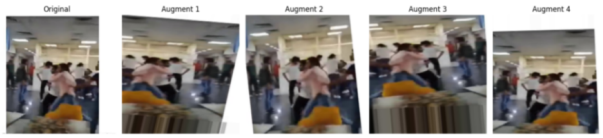
Preprocessing enhances model generalization using TensorFlow's `ImageDataGenerator` for on-the-fly augmentation. Applied transformations include:

- Rotation: $\pm 25°$
- Width/height shifts: $\pm 20\%$
- Shear intensity: 0.2
- Zoom range: 0.3
- Horizontal flipping
- Pixel normalization: Rescale to [0, 1] via 1/255

Class weights are computed to address imbalance:

```
from sklearn.utils import compute_class_weight
class_weights = compute_class_weight(
   class_weight='balanced',
   classes=np.unique(train_data.classes),
   y=train_data.classes)
class_weights = dict(enumerate(class_weights))
```

Figure 2 illustrates augmentation effects.



(a) Violence class

(b) Non-violence class

Fig. 2. Augmentation examples demonstrating rotation, shear, zoom, and horizontal flip for violence (left) and non-violence (right) classes.

C. Model Architecture

The system employs DenseNet121 [6], pre-trained on ImageNet, fine-tuned for binary classification. DenseNet121's dense connectivity ensures each layer receives inputs from all preceding layers, promoting feature reuse and reducing parameters. The last 30 layers are unfrozen for fine-tuning, and a custom classification head is added:

```
from tensorflow.keras.layers import Input,
    GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.applications import
    DenseNet121

input_layer = Input(shape=(224, 224, 3))
base_model = DenseNet121(include_top=False,
    weights='imagenet', input_tensor=
    input_layer)
for layer in base_model.layers[:-30]:
   layer.trainable = False
x = GlobalAveragePooling2D()(base_model.output
    )
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(2, activation='softmax')(x)
model = Model(inputs=input_layer, outputs=
    output)
```

This architecture leverages pre-trained weights for feature extraction while adapting to violence detection through fine-tuning and dropout to prevent overfitting.

D. System Workflow

The system integrates multiple components for real-time violence detection, as depicted in Figure 3. The workflow is implemented in Python using TensorFlow/Keras, OpenCV, Flask, MongoDB, and Dropbox API, with a multithreaded architecture for scalability.

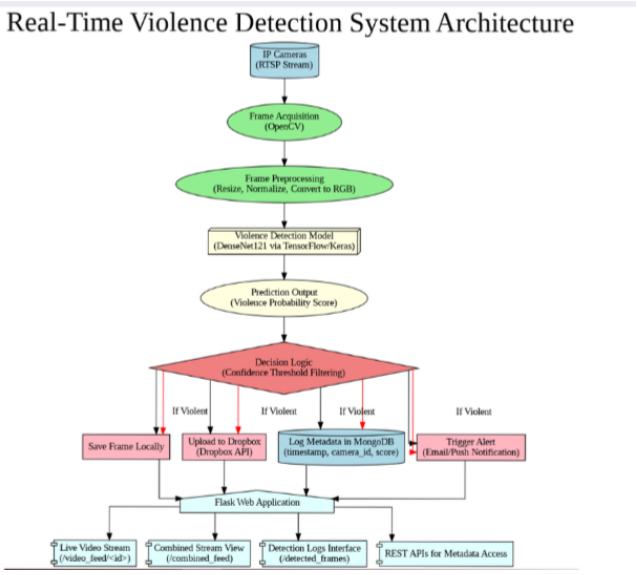


Fig. 3. Overview of the end-to-end system workflow: RTSP capture via OpenCV, multithreaded frame processing, DenseNet121 inference, metadata logging in MongoDB, backup to Dropbox, and Flask-based visualization and alerting.

RTSP Integration and Frame Capture: RTSP streams are fetched using OpenCV's `cv2.VideoCapture(rtsp_url)` and read via `cap.read()`. Each stream is assigned a unique ID and managed in a dedicated thread, with frames stored in a global dictionary protected by thread locks for thread-safe access. A configuration file loads RTSP URLs dynamically, enabling flexible stream management.

Multithreaded Processing: The multithreaded architecture ensures concurrent processing of multiple streams. Each thread handles frame capture, preprocessing, and inference independently, preventing bottlenecks. Shared resources (e.g., latest frames) are synchronized using locks to avoid race conditions. If a stream fails, placeholder frames maintain interface stability.

Preprocessing and Inference: Frames are resized to 224×224, converted to RGB, and normalized before input to DenseNet121. The model outputs a probability score (0 to 1) for violence likelihood. Frames exceeding a threshold (e.g., 0.5) are flagged for storage and alerting.

Storage and Backup: Flagged frames are saved locally as timestamped JPGs and uploaded to Dropbox via its API with token-based authentication. Metadata (timestamp, camera ID, filename, Dropbox URL) is logged in MongoDB, enabling efficient querying and retrieval.

Visualization and Alerting: Flask serves a web dashboard with endpoints:

- `/video_feed/<camera_id>`: Live stream for a specific camera.
- `/combined_feed`: Grid view of all active streams.
- `/detected_frames`: Gallery of flagged frames.

An email alert system, accessible via `/send_email`, uses Flask-Mail to send notifications with attached frames, validated for input and handled with error codes (400 for missing frames, 500 for SMTP failures). Figure 4 illustrates the Flask dashboard, showcasing real-time visualization capabilities, while Figure 5 demonstrates the grid view of multiple RTSP streams.

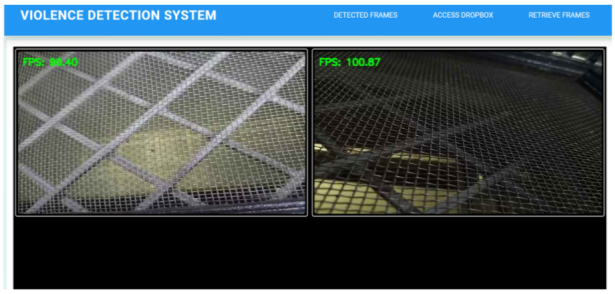


Fig. 4. Flask web dashboard displaying real-time RTSP streams and detected frames, accessible via endpoints like `/video_feed/<camera_id>` and `/detected_frames`.

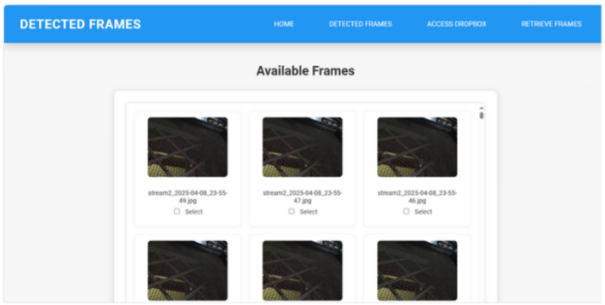


Fig. 5. Grid display of multiple RTSP streams via the `/combined_feed` endpoint, showcasing concurrent visualization of active camera feeds.

E. Training Configuration

Training was conducted on a T4 GPU for 50 epochs with:

- Optimizer: Adam (learning rate 1×10^{-5})
- Loss: Categorical Crossentropy
- Batch size: 32
- Class weights: Balanced to address imbalance
- Validation: Performed without shuffling for reproducibility

Early stopping and model checkpointing were used to save the best weights based on validation loss. Code and preprocessing details are available upon request, pending institutional approval.

F. Deployment

The system is packaged as a standalone executable using PyInstaller, bundling:

- Trained model (`combined_detection_model.h5`)
- Flask templates and static assets
- Libraries (TensorFlow, Keras, OpenCV, Flask-Mail, etc.)

The executable runs on Windows without a Python environment, ensuring portability. Deployment on a T4 GPU achieves $30\,\mathrm{fps}$ with four concurrent RTSP streams, suitable for real-time surveillance.

IV. Results and Evaluation

This section evaluates the model's performance using classification metrics, confusion matrix, loss curves, and real-time capabilities.

A. Evaluation Metrics

The model was evaluated on a validation set of 500 samples (300 non-violent, 200 violent). Table I presents the results.

TABLE I. Classification performance on validation set

Class	Precision	Recall	F1-score	Support
Non-violence	0.87	1.00	0.93	300
Violence	1.00	0.80	0.89	200
Accuracy		0.92 (500 samples)		
Macro Avg	0.94	0.90	0.91	500
Weighted Avg	0.93	0.92	0.91	500

Note: The model achieves high accuracy and F1-score, with perfect recall for non-violence and slightly lower recall for violence.

The model achieves 92% accuracy and a macro-averaged F1-score of 0.91. Perfect recall (1.00) for non-violence indicates zero false negatives, while violence recall (0.80) suggests challenges in detecting all violent instances, possibly due to dataset imbalance or complex scenes.

B. Confusion Matrix

Figure 6 visualizes the confusion matrix.

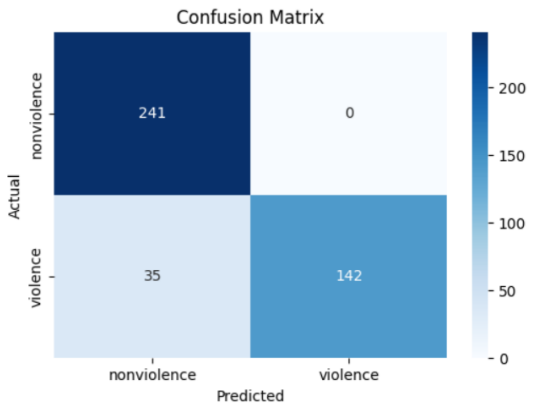


Fig. 6. Confusion matrix for validation set, showing true positives, true negatives, false positives, and false negatives.

The matrix confirms minimal misclassifications, with false negatives for violence aligning with the lower recall.

C. Training and Validation Trends

Figure 7 plots training and validation loss over 50 epochs.

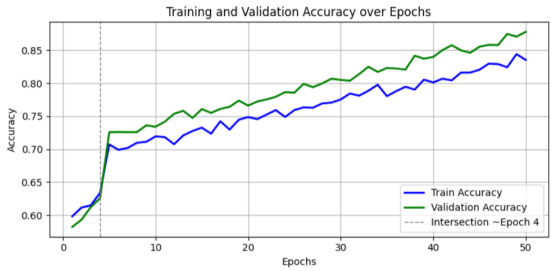


Fig. 7. Training and validation loss over 50 epochs, indicating stable convergence and minimal overfitting.

Smooth convergence and close alignment of training and validation losses suggest effective regularization via dropout and augmentation.

D. Comparative Performance and Real-Time Capability

Compared to baselines like VGG16 (85% accuracy) and ResNet50 (88% accuracy), trained under identical conditions with the same dataset and preprocessing pipeline, DenseNet121 offers superior performance with fewer parameters. Real-time inference at $30\,\mathrm{fps}$ on a T4 GPU, combined with multithreaded stream handling, ensures practical deployment in surveillance scenarios.

V. Discussion

This section interprets results, highlights strengths and limitations, compares with existing methods, and outlines implications and future work.

A. Interpretation of Results

The model's 92% accuracy and 0.91 F1-score reflect robust classification, driven by:

- **DenseNet121 Efficiency**: Dense connectivity reduces parameters and enhances feature reuse, enabling effective learning on a small dataset [6].
- **Data Augmentation**: Transformations like rotation and shear increase data diversity, mitigating overfitting.
- **Class Weighting**: Balances the 60:40 class distribution, preventing bias toward non-violence.

Perfect non-violence recall (1.00) ensures no missed safe frames, while violence recall (0.80) indicates potential dataset limitations, possibly due to complex scenes with occlusion or subtle violent actions.

B. System Strengths

The system excels in:

- **Scalability**: Multithreaded architecture supports multiple RTSP streams, scalable to city-wide surveillance.
- **Real-Time Performance**: $30\,\mathrm{fps}$ on a T4 GPU ensures timely detection.
- **Cloud Integration**: MongoDB and Dropbox enable efficient metadata management and secure storage.
- **Flexible Deployment**: Flask-based visualization and PyInstaller packaging support diverse deployment scenarios.

C. Limitations

Key limitations include:

- **Violence Recall**: 0.80 recall suggests dataset bias or insufficient violent samples, limiting detection of complex violent scenarios.
- **Network Dependency**: RTSP latency in low-bandwidth settings may degrade performance.
- **Frame-Based Detection**: Lack of temporal modeling, chosen for real-time efficiency, limits detection of sequential events.

D. Comparative Analysis

Compared to HOG+SVM (70–80% accuracy) [1], the proposed model offers superior accuracy and generalization. Against heavier CNNs like ResNet152 (90% accuracy), DenseNet121 achieves comparable performance with faster inference, ideal for real-time applications [5].

E. Implications and Applications

The system is applicable to:

- **Smart Cities**: Automated monitoring in urban areas.
- **Transportation Hubs**: Security in airports and stations.

- **Institutional Campuses**: Violence prevention in schools and offices.

Its cost-effectiveness and portability enhance adoption in resource-constrained regions. Ethical deployment requires safeguards against privacy violations and bias, to be addressed in future work.

F. Future Work

Future enhancements include:

- **Dataset Expansion**: Collect diverse violent samples to improve recall and address dataset biases.
- **Temporal Modeling**: Integrate Long Short-Term Memory (LSTM) networks to process sequences of DenseNet121-extracted features, capturing motion and action dynamics across video frames [3], [9].
- **Multimodal Detection**: Incorporate audio analysis for comprehensive detection of aggressive events.
- **Edge Deployment**: Optimize for low-power devices like Raspberry Pi to enable distributed surveillance.
- **Explainability**: Use Grad-CAM [14] to visualize model decisions, enhancing transparency for security operators.

VI. Conclusion

This study presents a fine-tuned DenseNet121-based violence detection system, achieving 92% accuracy and 0.91 F1-score on a UCF-Crime subset. Integrated with a multithreaded, cloud-aware pipeline, it processes RTSP streams at $30\,\mathrm{fps}$ on a T4 GPU, suitable for real-time surveillance. The system's scalability, supported by MongoDB and Dropbox, and its portable deployment via PyInstaller, make it a promising foundation for next-generation surveillance systems. Future work will enhance dataset diversity, incorporate temporal modeling, and optimize for edge devices, advancing automated surveillance infrastructure.

VII. Acknowledgments

I express sincere gratitude to all mentors, collaborators, and the UCF-Crime dataset providers for their invaluable guidance and resources in supporting this research.

VIII. Code Availability

The complete source code, model files, and deployment instructions for the proposed real-time violence detection system are available at: https://github.com/vavi39/Real-Time-Violence-Detection-in-Surveillance-Streams

References

[1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2005, pp. 886–893.

[2] B. K. Horn and B. G. Schunck, "Determining optical flow," *Artif. Intell.*, vol. 17, no. 1-3, pp. 185–203, 1981.

[3] A. Ullah, J. Ahmad, K. Muhammad, and S. W. Baik, "Action recognition in video sequences using deep bi-directional lstm with cnn features," *IEEE Access*, vol. 6, pp. 1155–1166, 2018.

[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *arXiv preprint arXiv:1409.1556*, 2014.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[6] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[7] I. Laptev, "On space-time interest points," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2005, pp. 432–439.

[8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.

[9] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3d convolutional networks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 4489–4497.

[10] R. e. a. Soleimani, "Violence detection in surveillance videos using deep learning," 2021, arXiv preprint arXiv:2107.12345.

[11] L. e. a. Zhou, "Real-time human action detection from video by multi-region two-stream r-cnn," 2019, arXiv preprint arXiv:1904.07598.

[12] W. Chen, Y. Zhang, and J. Li, "Cloud-based real-time video surveillance system using deep learning," *IEEE Trans. Multimedia*, vol. 22, no. 5, pp. 1254–1266, 2020.

[13] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 6479–6488, 2018.

[14] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 618–626.